جامعة خليفة
Khalifa University

Matplotlib – Plotting with Python

Dr. Sergio Martinez

Course layout

1. Introduction
2. Figures and axes
3. Formatting
4. 2D plots
5. 3D plots
6. Save plots

1. Introduction

- **Matplotlib**
- Import the package
- pyplot API
- Plot backend

- **Matplotlib** is one of the most popular Python packages used for data visualization. Together with **NumPy** (numerical Python), it can be used as a replacement for **Matlab**.
- It provides an **object-oriented API** that helps in embedding plots in applications using Python GUI toolkits such as PyQt, WxPythonotTkinter.
- It can be used in Python and IPython shells, **Jupyter notebook** and web application servers.
- Matplotlib is written in Python and makes use of **NumPy**.
- At the moment of creating this training course, **Matplotlib Version 3.2.2 was used**. https://matplotlib.org/3.2.2/contents.html

1. Introduction

- Matplotlib
- **Import the package**
- pyplot API
- Plot backend

- To begin with Matplotlib, the **Pyplot** module from Matplotlib package is imported, with an alias **plt** as a matter of convention.

```
import matplotlib.pyplot as plt
```

- **matplotlib.pyplot** is a collection of command style functions that make Matplotlib work like Matlab. Each Pyplot function makes some change to a figure.

1. Introduction

- Matplotlib
- Import the package
- **pyplot API**
- Plot backend

- **pyplot** is a module of Matplotlib.
- It is collection of command style functions that make Matplotlib work like Matlab. Each Pyplot function makes some change to a figure:
  - Creates a figure.
  - Creates a plotting area in a figure.
  - Plots some lines in a plotting area.
  - Decorates the plot with labels.
- pyplot is **stateful**, in that it keeps track of the current figure and plotting area, and the plotting functions are directed to the current axes
- **Let's see some examples:**

  01_Introduction/pyplot_stateful.ipynb

1. Introduction

- Matplotlib
- Import the package
- pyplot API
- **Plot backend**

- It is possible to change the behavior of the notebook when the plots are created. This is achieved by selecting the **plot backend**.
- To set this up, before any plotting or import of matplotlib is performed you must execute the **%matplotlib** magic command:
  - **`%matplotlib inline`**: will draw static images in the notebook.
  - **`%matplotlib notebook`**: will lead to interactive plots embedded within the notebook, you can zoom and resize the figure.
- **Let's see some examples:**

  01_Introduction/plot_backend.ipynb

2. Figures and axes

- **Object-oriented API**
- Figure class
- Axes class

- At its core Matplotlib is **object-oriented**.
- Most of the functions available in **matplotlib.pyplot** are also available in the **matplotlib.axes.Axes** class.
- Working directly with the objects gives more **control and customization** to the plots.
- In object-oriented interface, Pyplot is used only for a few functions such as **figure creation**, and **the user explicitly creates and keeps track of the figure and axes objects**.
- **Let's see some examples:**

  02_Figures_and_axes/pyplot_object_oriented.ipynb

2. Figures and axes

- Object-oriented API
- **Figure class**
- Axes class

- The **matplotlib.figure** module contains the Figure class. It is a top-level container for all plot elements. The Figure object is instantiated by calling the **figure()** function from the pyplot module.

```
fig = plt.figure()
```

Sr.No.	Parameter & Description
1	**num** : integer or string, optional, default: None If not provided, a new figure will be created. If num is provided, and a figure with this id already exists, make it active, and returns a reference to it. If this figure does not exists, create it and returns it.
2	**figsize** : (float, float), optional, default: None Width, height in inches.
3	**dpi** : integer, optional, default: None Resolution of the figure.
4	**facecolor** : color spec The background color
5	**edgecolor** : color spec The border color.
6	**linewidth** : float The linewidth of the frame (i.e. the edge linewidth of the figure patch).

- **Let's see some examples:**

  02_Figures_and_axes/figures.ipynb

1. Introduction

- Object-oriented API
- Figure class
- **Axes class**

- **Axes** object is the region of the image with the data space.
- A given figure can contain many Axes, but a given Axes object **can only be in one** Figure.
- The Axes contains two (or three in the case of 3D) **Axis objects**.
- It contain most of the figure elements: Axis, Tick, Label, Legend, Text…
- The Axes class and its member functions are **the primary entry point to working with the OO interface**.
- There are multiple constructors to create axes objects. The most logical way is to **add axes to a figure using Figure functions**.

2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation I, **figure.add_axes()**:
  - The calling signature of add_axes is **add_axes(rect)**, where rect is a list [x0, y0, width, height] denoting the lower left point of the new axes in figure coordinates (x0,y0) and its width and height. So the axes is positioned in absolute coordinates on the canvas. E.g.

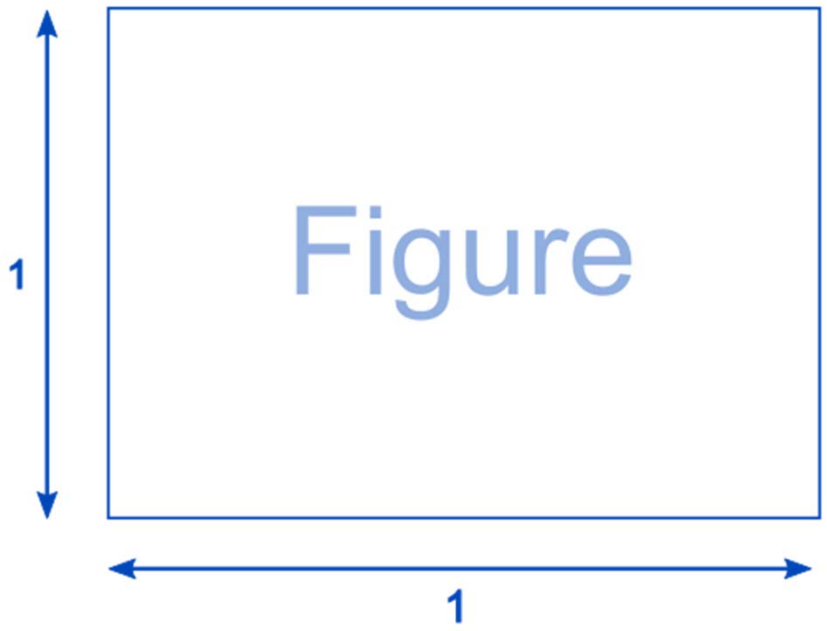
```
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
```

  - The values of the parameters are based on the normalized size of the figure.

2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

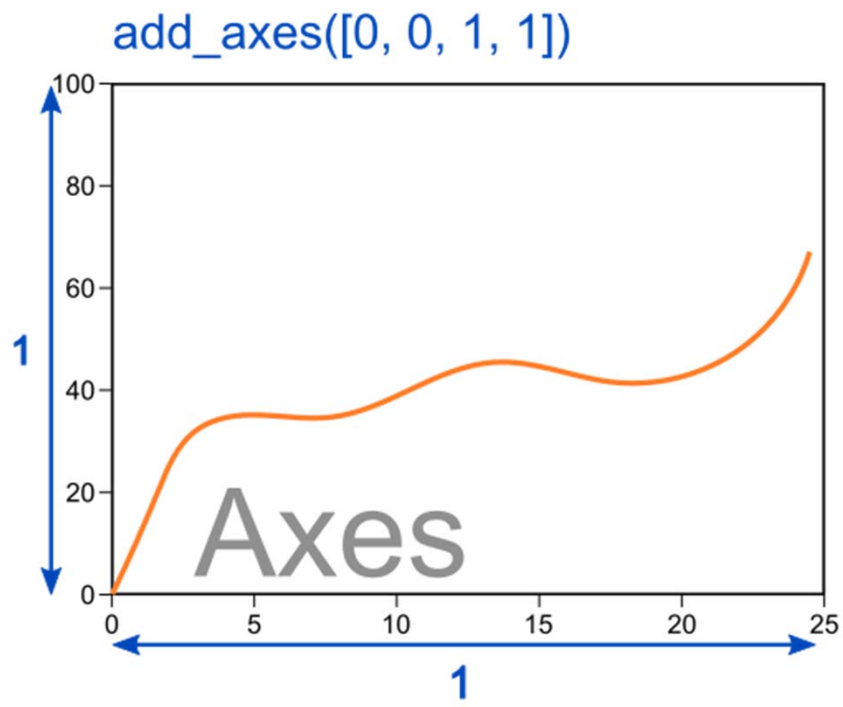
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

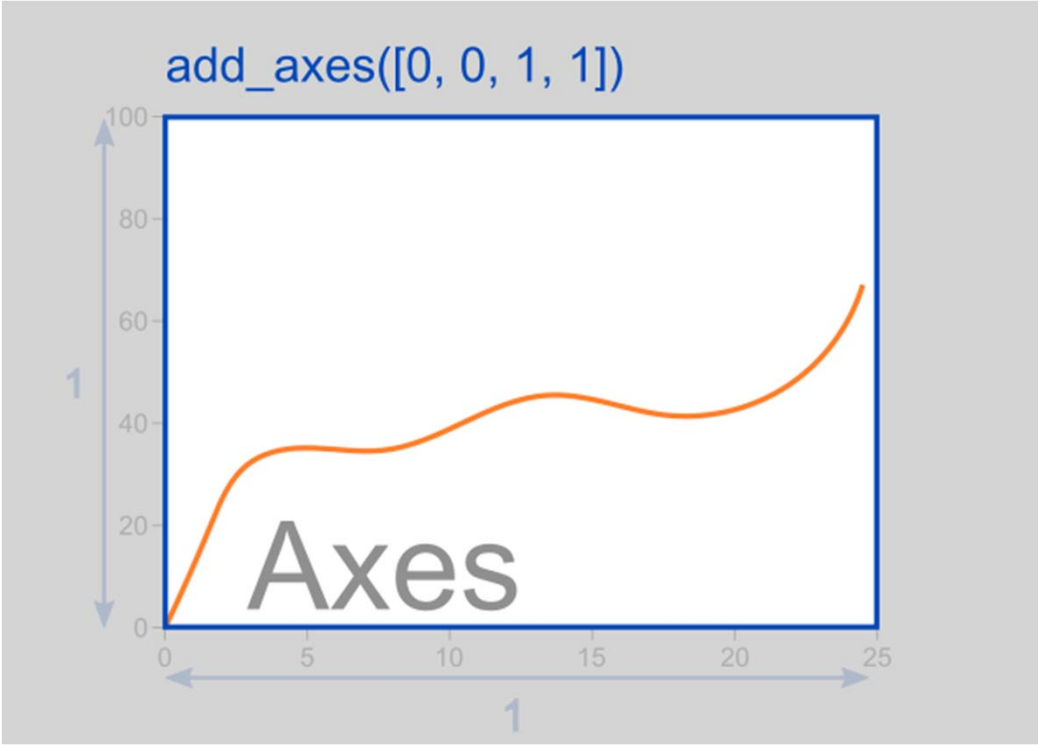
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

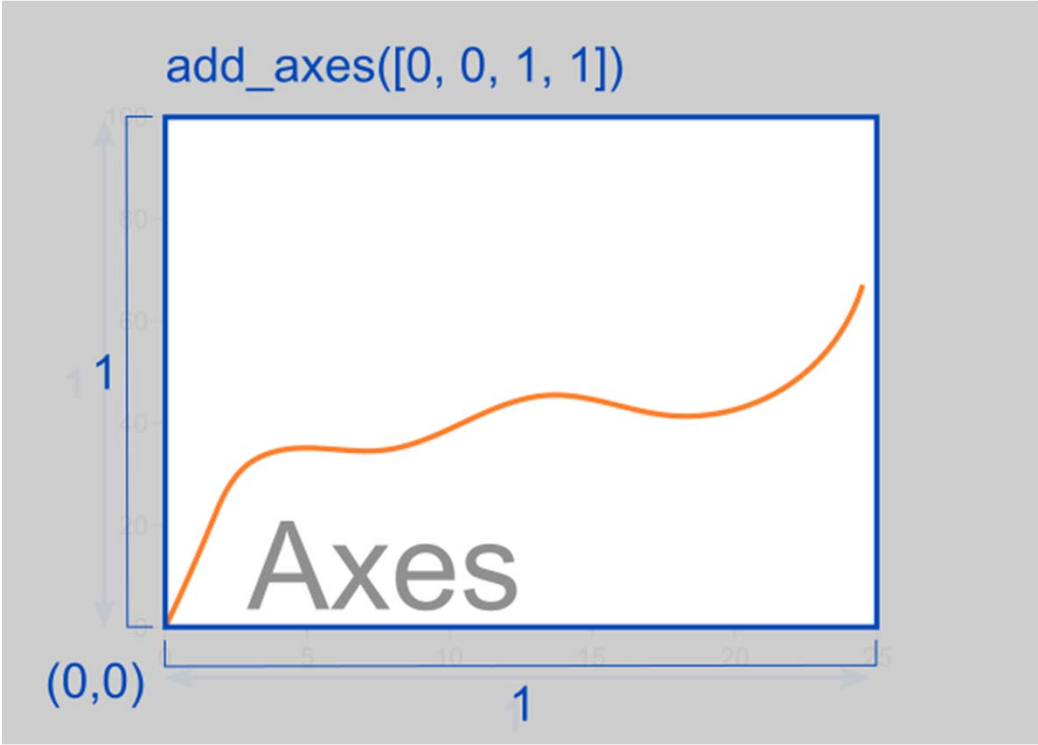
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

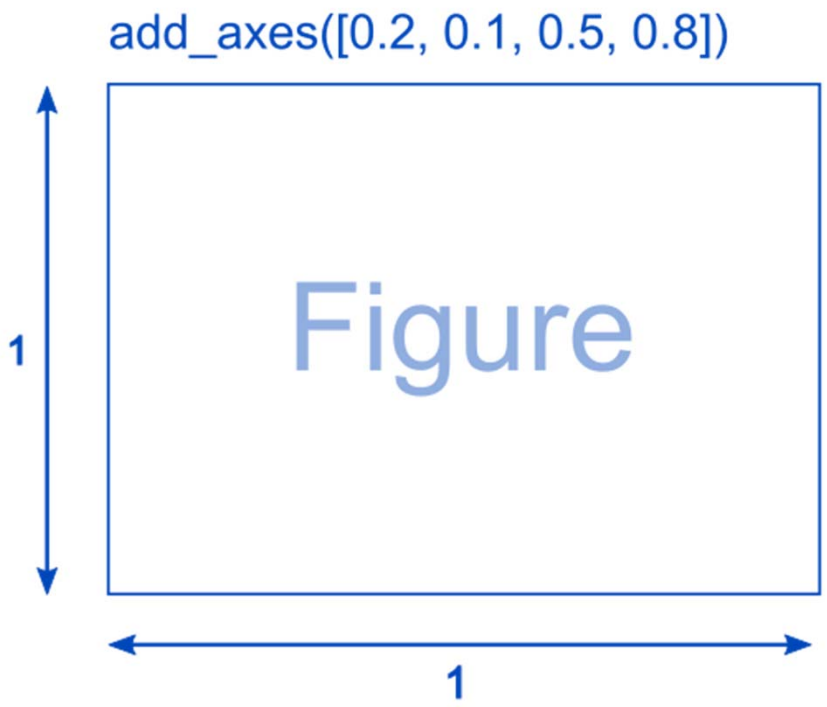
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

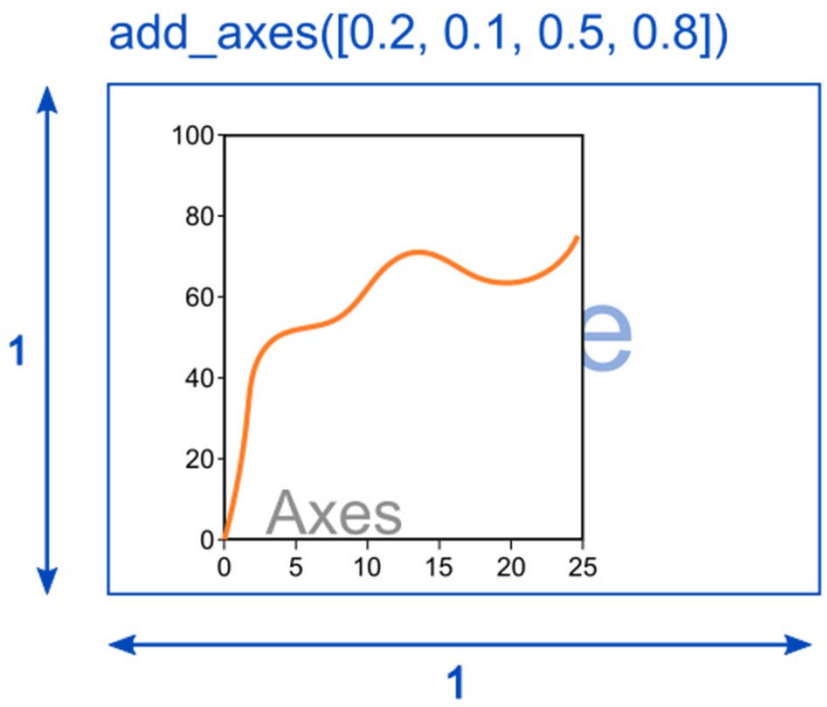
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

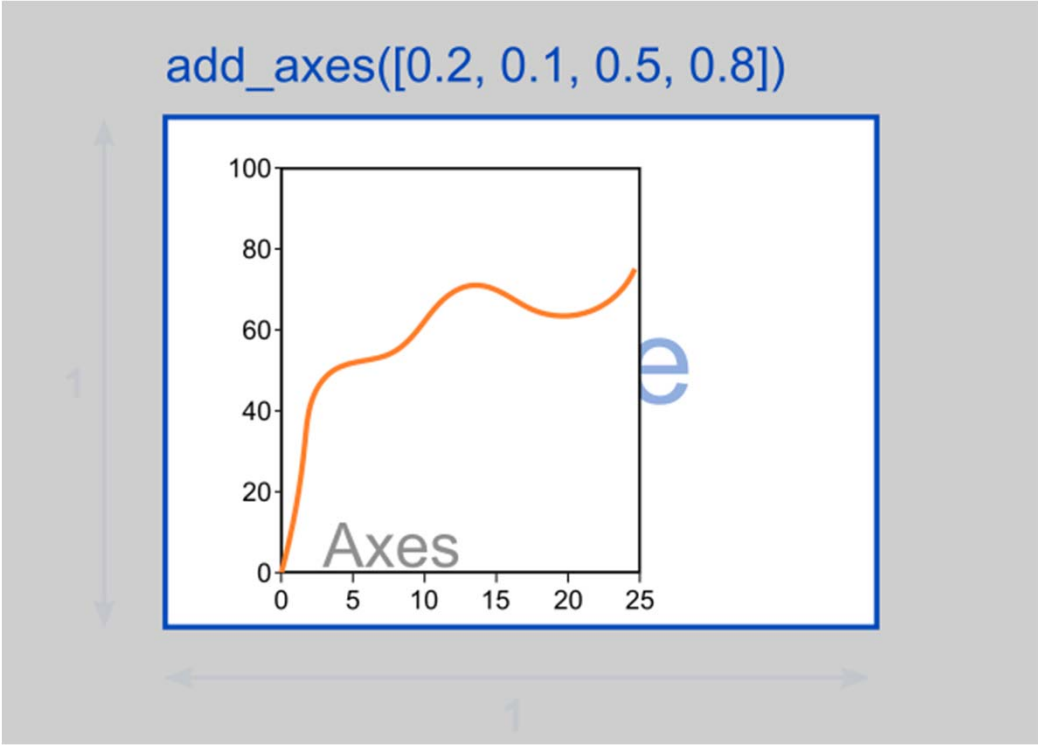
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

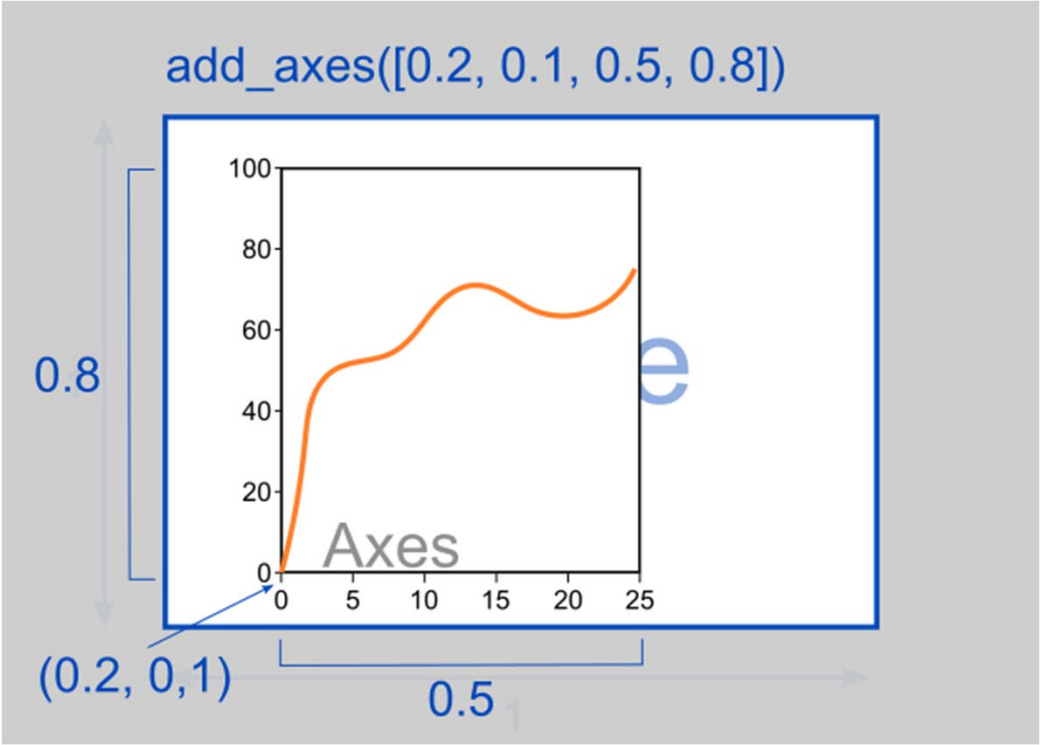
- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation I, **figure.add_axes()**:



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:
  - The calling signature of add_subplot is **add_subplot(nrows, ncols, index)**, it allows to specify where the axes should be situated according to a subplot grid.
  - The subplot will take the **index** position on a grid with **nrows** rows and **ncols** columns. index starts at 1 in the upper left corner and increases to the right.

```
fig = plt.figure()
ax = fig.add_subplot(2,3,1)
```

2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:
  - Another calling of add_subplot is **add_subplot(nrows,ncols, (first,last))**, it allows to specify where the axes should be situated according to a subplot grid.
  - It spans the subplot from **first** cell to **last** cell on a grid with **nrows** rows and **ncols** columns. index starts at 1 in the upper left corner and increases to the right.

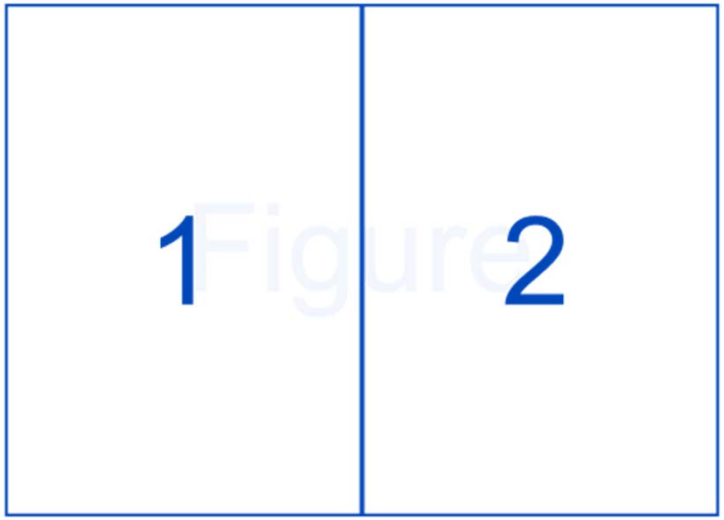
```
fig = plt.figure()
ax = fig.add_subplot(2,3,(2,5))
```

2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:

add_subplot(1,2,2)

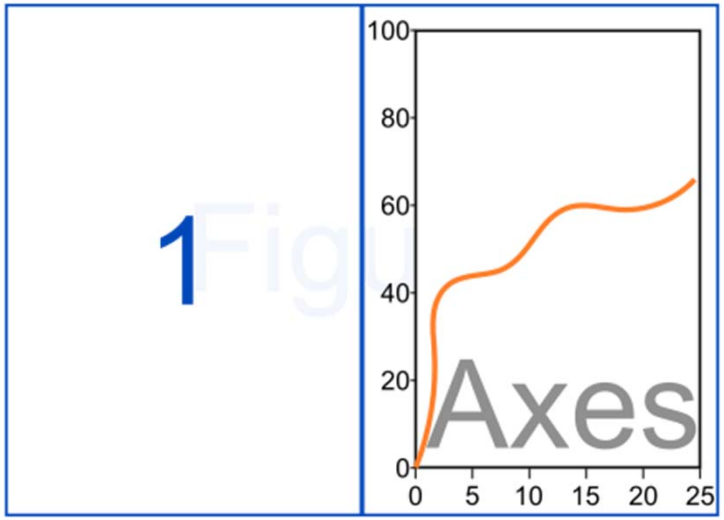


2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:

add_subplot(1,2,2)

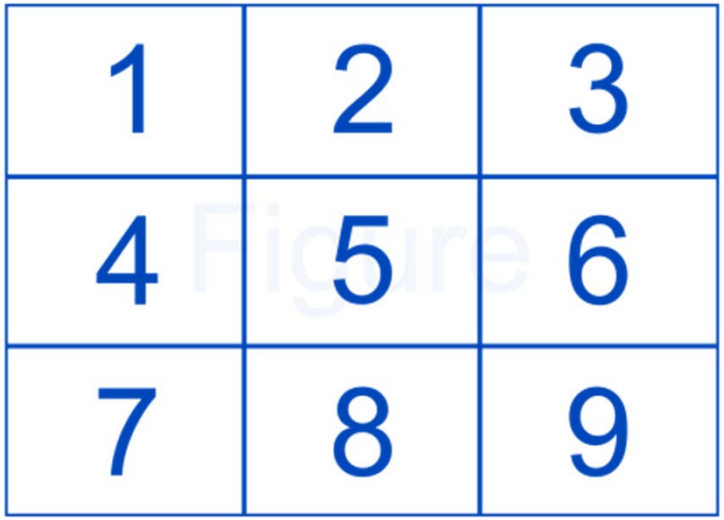


2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:

add_subplot(3,3,(4,9))

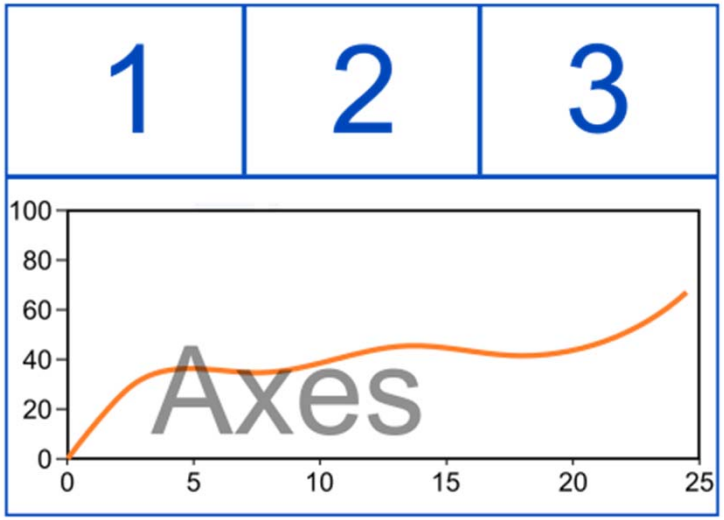


2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation II, **figure.add_subplot()**:

add_subplot(3,3,(4,9))



2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation III, **other functions to create axes**
  - **plt.subplot(nrows, ncols, index)**: In the current figure, the function creates and returns an Axes object, at position **index** of a grid of **nrows** by **ncols** axes. Creating a subplot will delete any pre-existing subplot that overlaps with it beyond sharing a boundary.
  - **plt.subplots(nrows, ncols)**: The two integer arguments to this function specify the number of **rows** and **columns** of the subplot grid. The function **returns a figure object and a tuple** containing axes objects equal to nrows*ncols. Each axes object is accessible by its index.

2. Figures and axes

- Object-oriented API
- Figure class
- **Axes class**

- Axes creation III, **other functions to create axes**
  - **plt. subplot2grid(shape,location,rowspan,colspan)**: This function gives more flexibility in creating an axes object at a specific location of the grid. It also allows the axes object to be spanned across multiple rows or columns.

- **Let's see some examples:**

  02_Figures_and_axes/axes.ipynb

3. Formatting

- **Grids**
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- Legend

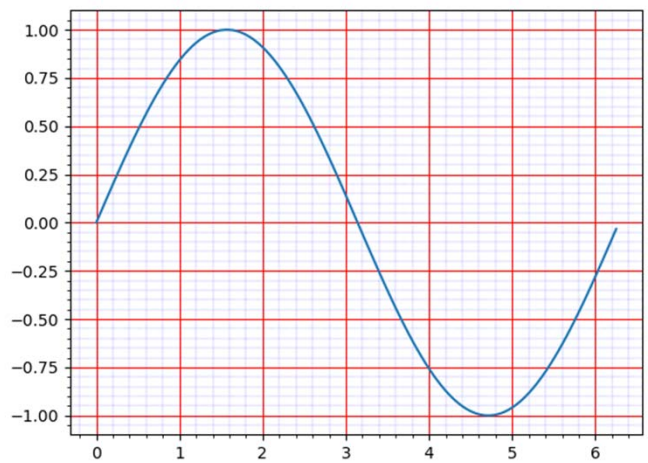
- The **grid()** function of axes object sets visibility of grid inside the figure to on or off. You can also display major / minor (or both) ticks of the grid. Additionally **color, linestyle and linewidth** properties can be set in the grid() function.

Sr.No.	Parameter & Description
1	**which**{'major', 'minor', 'both'}, optional The grid lines to apply the changes on.
2	**axis**{'both', 'x', 'y'}, optional The axis to apply the changes on.
3	**color** color
4	**linestyle** {'-', '--', '-.', ':', '', (offset, on-off-seq), ...}
5	**linewidth** float

3. Formatting

- **Grids**
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- Legend

- The **grid()** function of axes object sets visibility of grid inside the figure to on or off. You can also display major / minor (or both) ticks of the grid. Additionally **color, linestyle and linewidth** properties can be set in the grid() function.



- **Let's see some examples:**

  03_Formatting/grids.ipynb

3. Formatting

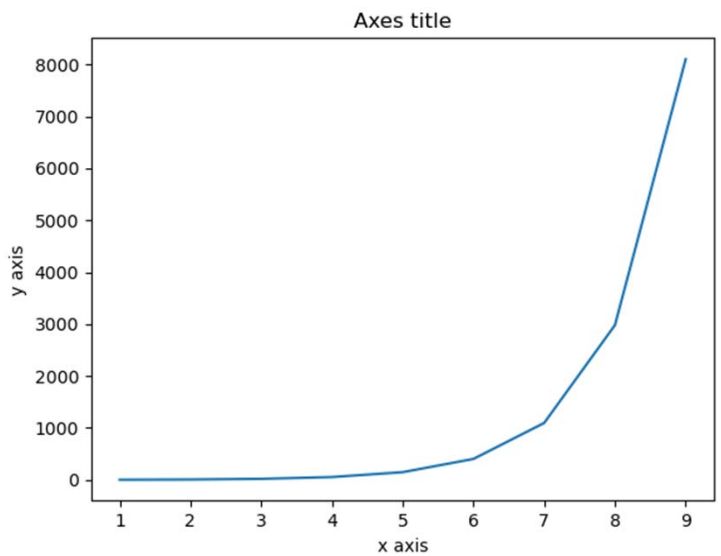
- Grids
- **Labels**
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- Legend

- It is always desirable to include labels on the plot:
  - **Axes.set_title()**: sets a title for the axes.
  - **Axes.set_xlabel()**: sets label of x axis.
  - **Axes.set_ylabel()**: sets label of y axis.

3. Formatting

- Grids
- **Labels**
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- Legend

- Axes labeling:



- **Let's see some examples:**

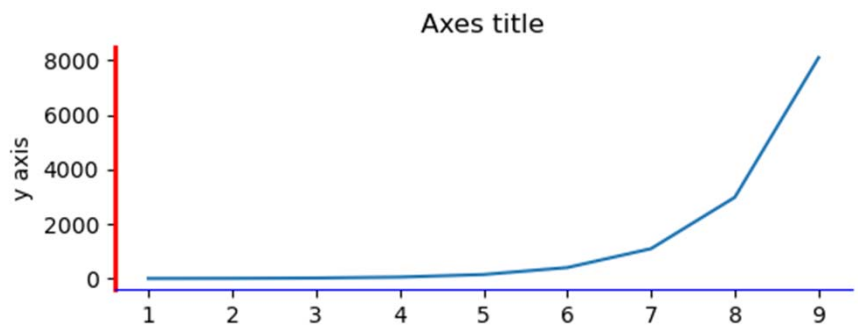
  03_Formatting/labels.ipynb

3. Formatting

- Grids
- Labels
- **Spines**
- Scale
- Limits
- Ticks
- Twin axes
- Legend

- Axis **spines** are the lines connecting axis tick marks demarcating boundaries of plot area. The axes object has spines located at **top**, **bottom**, **left** and **right**.

- Each spine can be formatted by specifying color and width. Any edge can be made **invisible** if its color is set to none.



- **Let's see some examples:**

  03_Formatting/spines.ipynb

3. Formatting

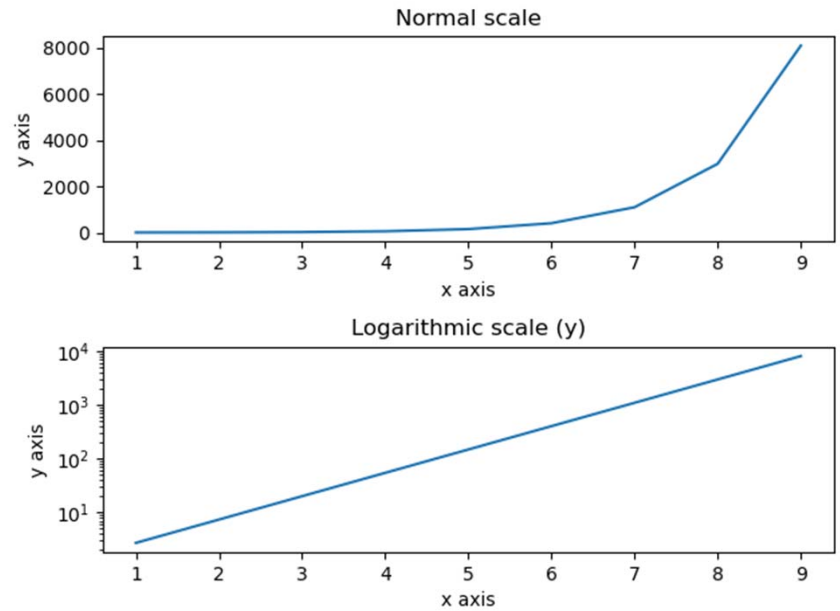
- Grids
- Labels
- Spines
- **Scale**
- Limits
- Ticks
- Twin axes
- Legend

- Sometimes, one or a few points are much larger than the bulk of data. In such a case, the **scale** of an axis needs to be set as logarithmic rather than the normal scale. In Matplotlib, it is possible by setting **xscale** or **yscale** property of axes object to ‘log’.
  - **Axes.set_xscale("log")**: sets logarithmic scale in x-axis.
  - **Axes.set_yscale("log")**: sets logarithmic scale in y-axis.

3. Formatting

- Grids
- Labels
- Spines
- **Scale**
- Limits
- Ticks
- Twin axes
- Legend

- Axes scale:



- **Let's see some examples:**

  03_Formatting/scale.ipynb

3. Formatting

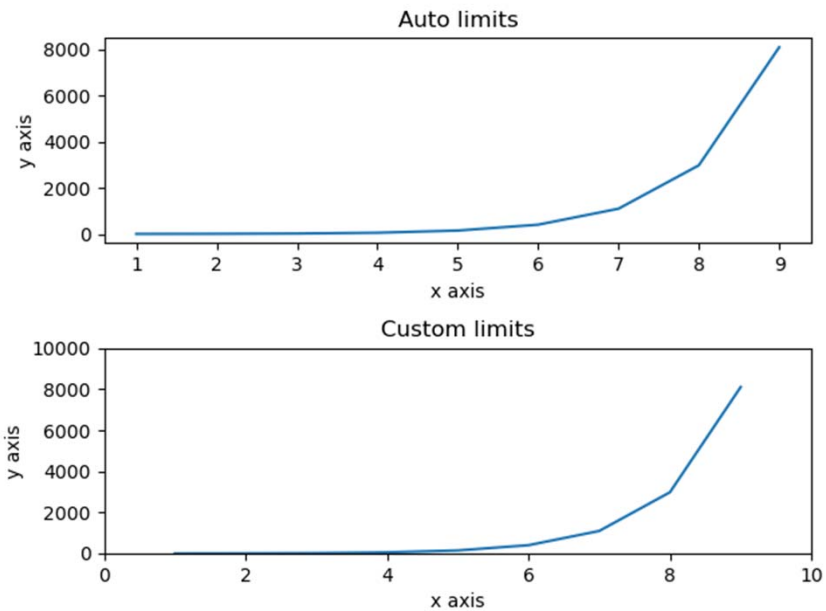
- Grids
- Labels
- Spines
- Scale
- **Limits**
- Ticks
- Twin axes
- Legend

- Matplotlib automatically arrives at the minimum and maximum values of variables to be displayed along x, y (and z axis in case of 3D plot) axes of a plot. However, it is possible to set the limits explicitly by using **set_xlim()** and **set_ylim()** functions.
  - **Axes.set_xlim(left, right)**: Set the x-axis view limits.
  - **Axes.set_ylim(bottom,top)**: Set the y-axis view limits.

3. Formatting

- Grids
- Labels
- Spines
- Scale
- **Limits**
- Ticks
- Twin axes
- Legend

- Limiting axes view:



- **Let's see some examples:**

  03_Formatting/limits.ipynb

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- **Ticks**
- Twin axes
- Legend

- **Ticks** are the markers denoting data points on axes. Matplotlib's default tick locators and formatters are designed to be generally sufficient in many common situations. Position and labels of ticks can be explicitly mentioned to suit specific requirements.

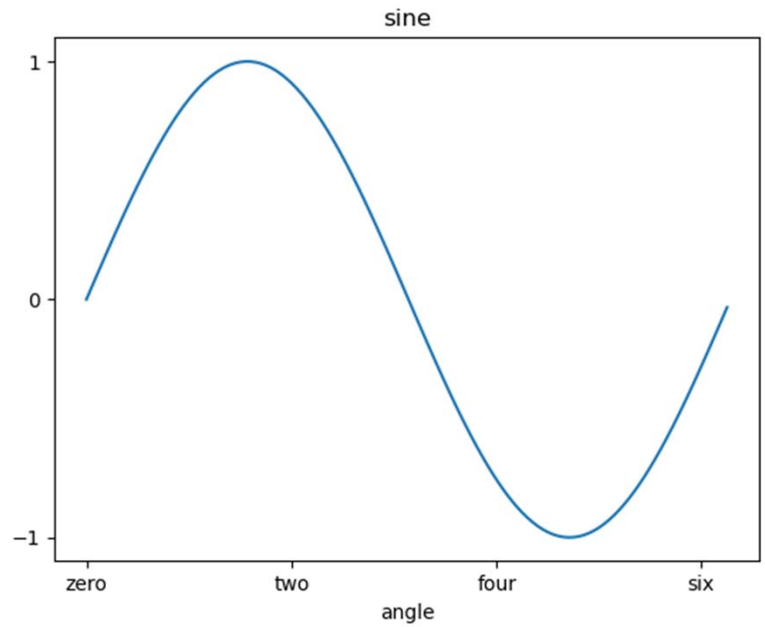
- The **set_xticks()** and **set_yticks()** function takes a list object as argument. The elements in the list denote the positions on corresponding action where ticks will be displayed.

- Similarly, labels corresponding to tick marks can be set by **set_xlabels()** and **set_ylabels()** functions respectively.

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- **Ticks**
- Twin axes
- Legend

- Ticks and ticks' labels:



- **Let's see some examples:**

  03_Formatting/ticks.ipynb

3. Formatting

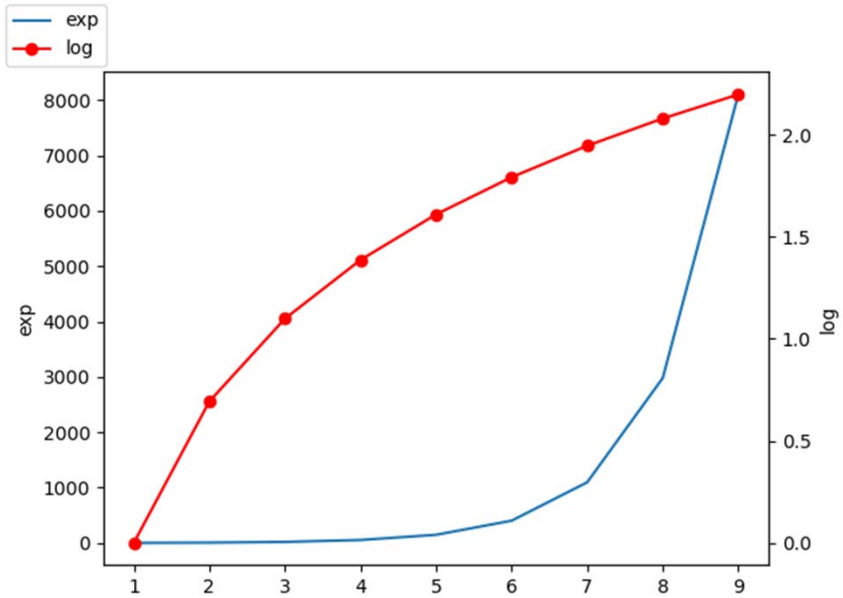
- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- **Twin axes**
- Legend

- It is useful to have dual x or y axes in a figure. More so, when plotting curves with different units together. Matplotlib supports this with the **twinx** and **twiny** functions.
  - **Axes.twinx()**: Create a twin Axes sharing the x-axis.
  - **Axes.twiny()**: Create a twin Axes sharing the y-axis.

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- **Twin axes**
- Legend

- Twin axes:



- **Let's see some examples:**

  03_Formatting/twin_axes.ipynb

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

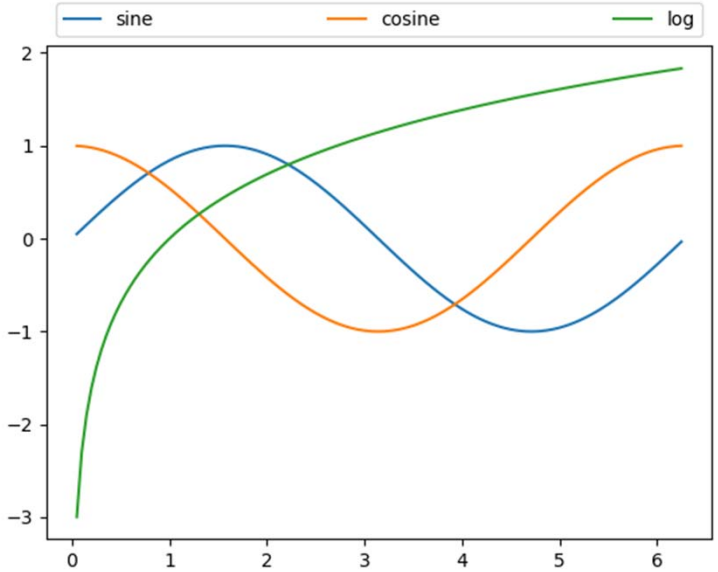
- The **legend(handles, labels, loc)** method of axes class adds a legend to the plot figure. It takes three parameters:
  - handles: a sequence of Line2D or Patch instances.
  - labels: a sequence of strings.
  - loc: can be a string or an integer specifying the legend location.

Location String	Location Code
'best'	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend:



3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

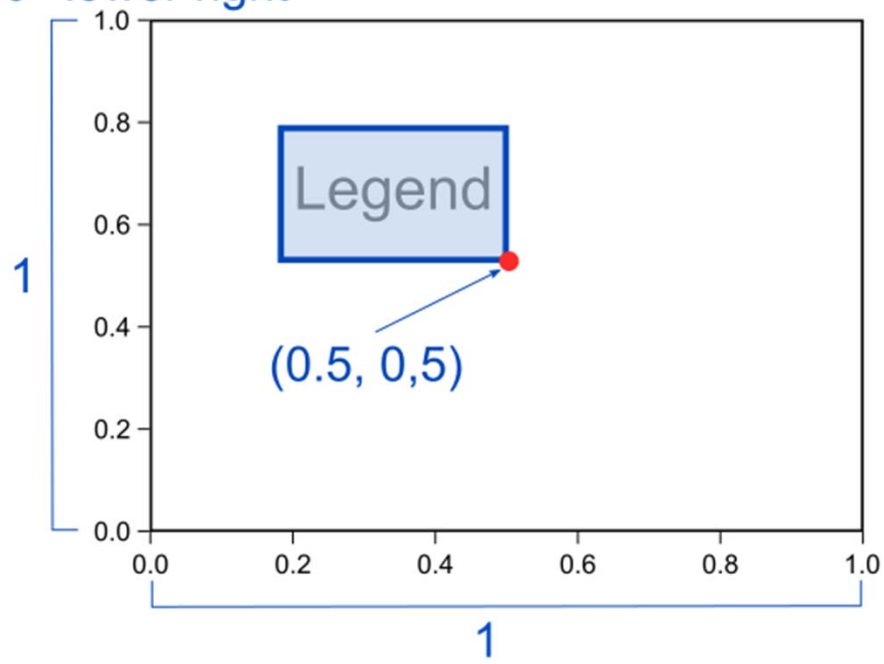
- In order to have more control on the legend's position and size, **bbox_to_anchor** parameter can be used together with **loc**. Following is the syntax:
  - **bbox_to_anchor=(x, y)**: Places the corner of the legend specified by *loc* at x, y.
  - **bbox_to_anchor=(x, y, width, height)**. Specifies the box where the legend is placed in.

3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y)**

bbox_to_anchor=(0.5, 0.5)
loc='lower right'

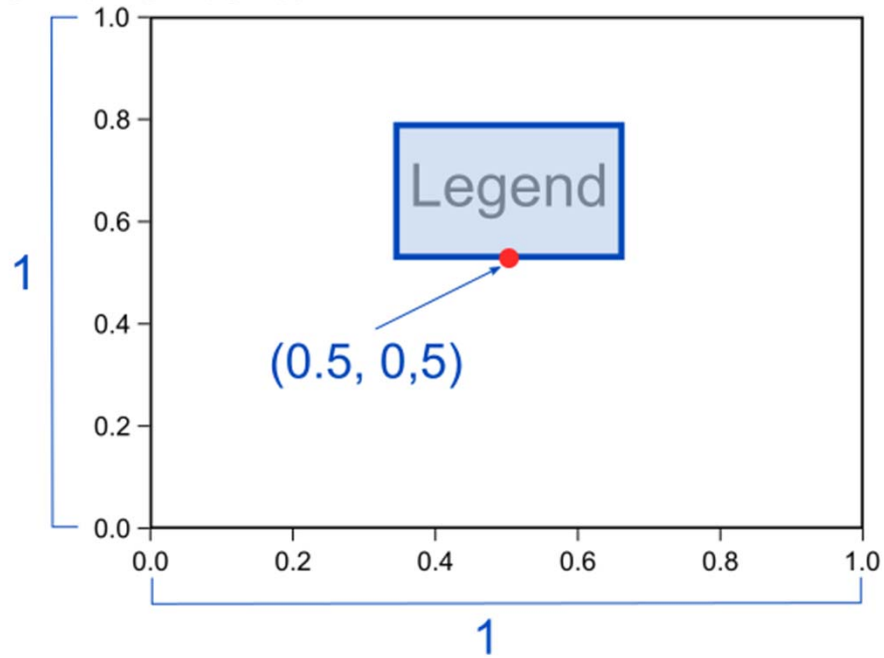


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y)**

bbox_to_anchor=(0.5, 0.5)
loc='lower center'

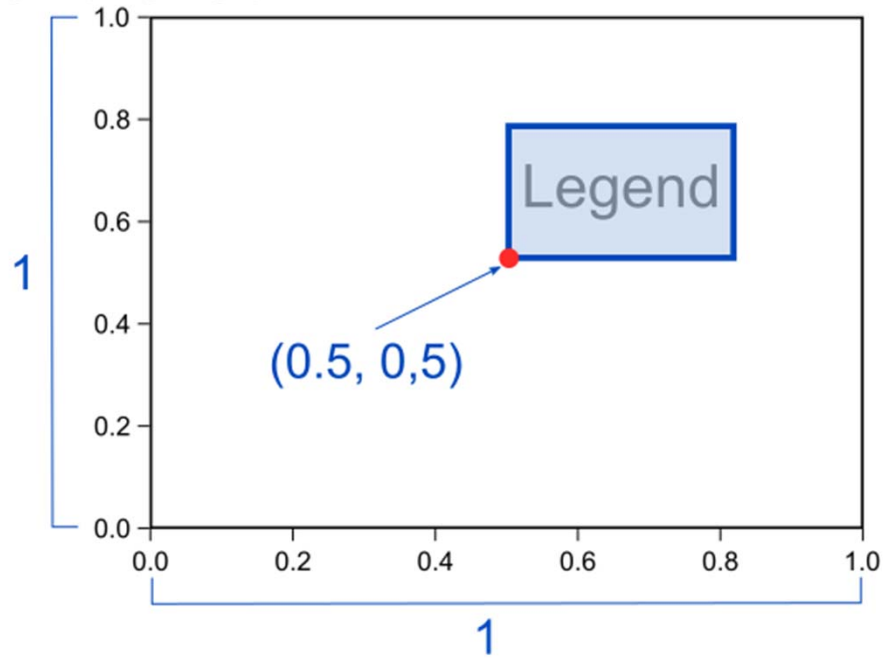


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y)**

bbox_to_anchor=(0.5, 0.5)
loc='lower left'

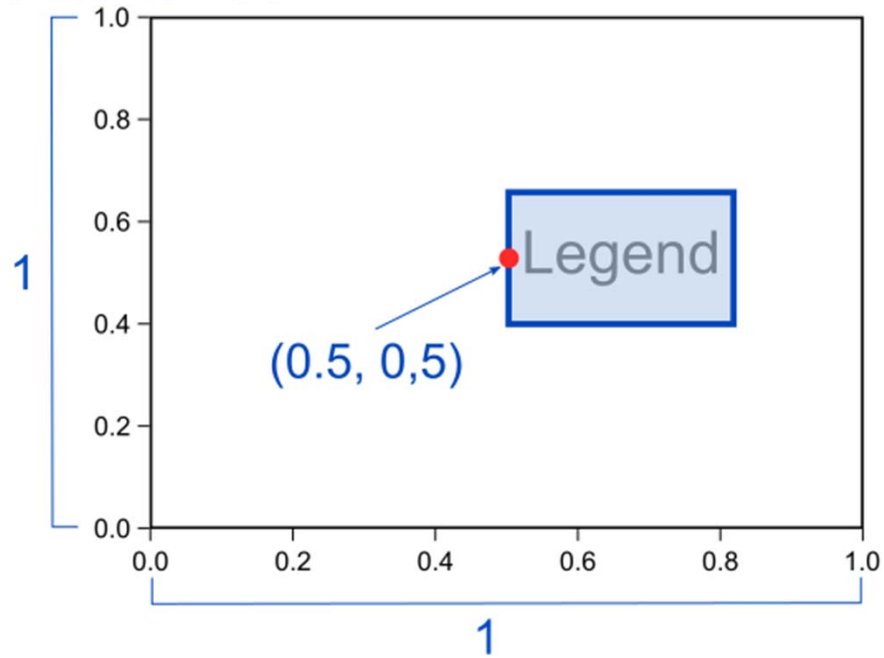


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y)**

bbox_to_anchor=(0.5, 0.5)
loc='center left'

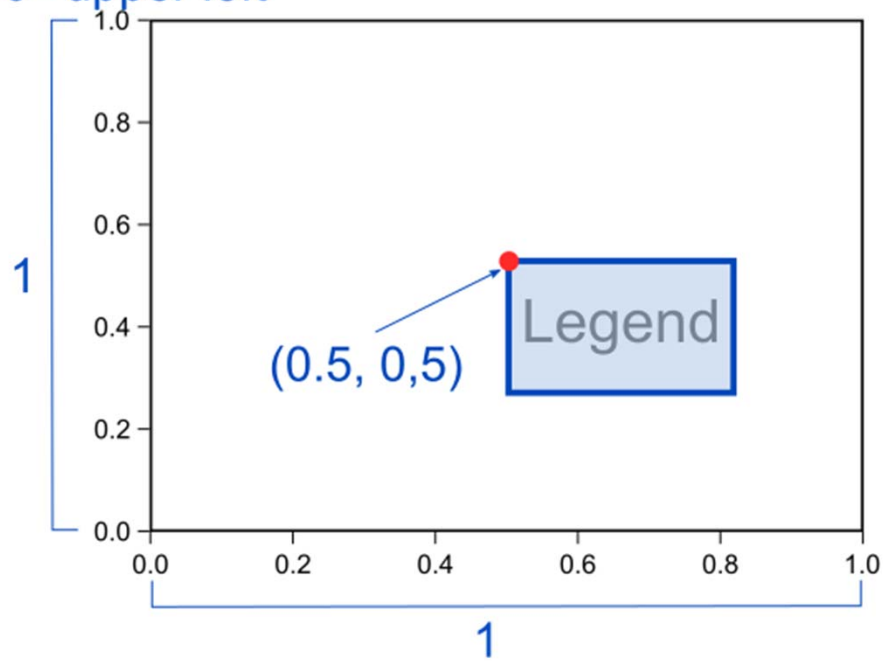


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y)**

bbox_to_anchor=(0.5, 0.5)
loc='upper left'

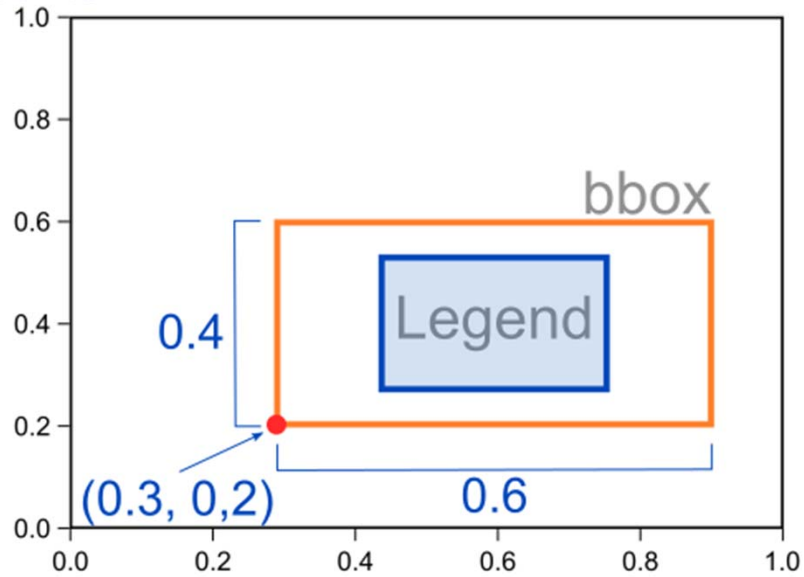


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**

bbox_to_anchor=(0.3, 0.2, 0.6, 0.4)
loc='center'

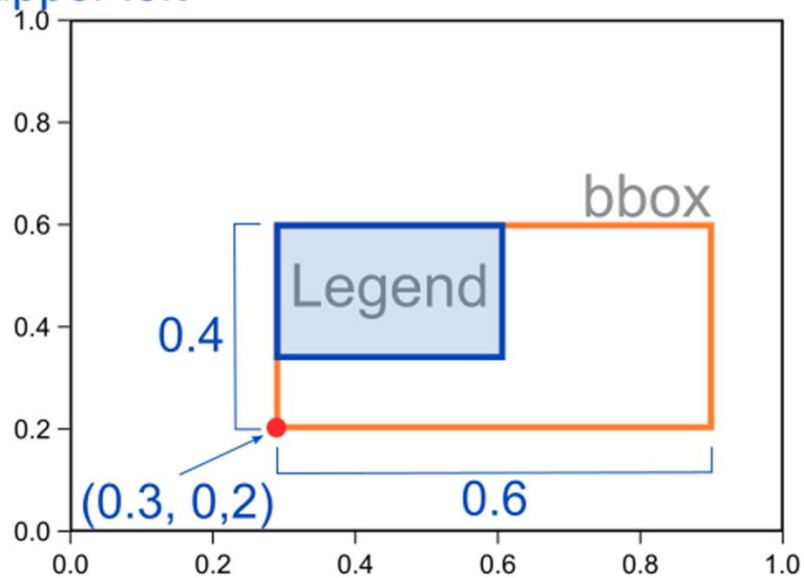


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**

bbox_to_anchor=(0.3, 0.2, 0.6, 0.4)
loc='upper left'

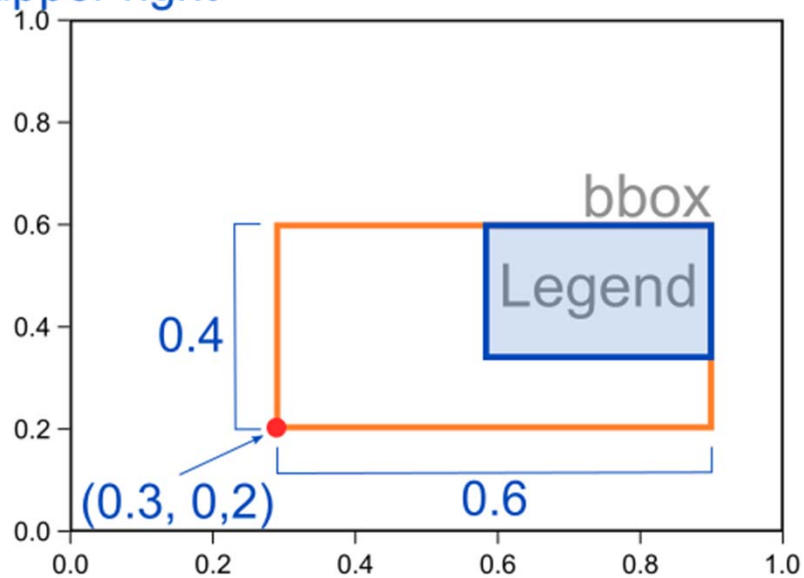


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**

bbox_to_anchor=(0.3, 0.2, 0.6, 0.4)
loc='upper right'

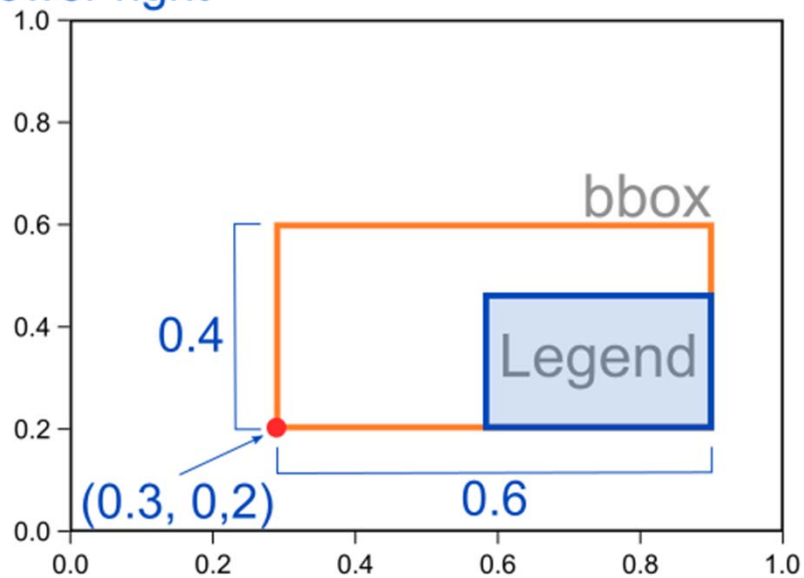


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**

bbox_to_anchor=(0.3, 0.2, 0.6, 0.4)
loc='lower right'

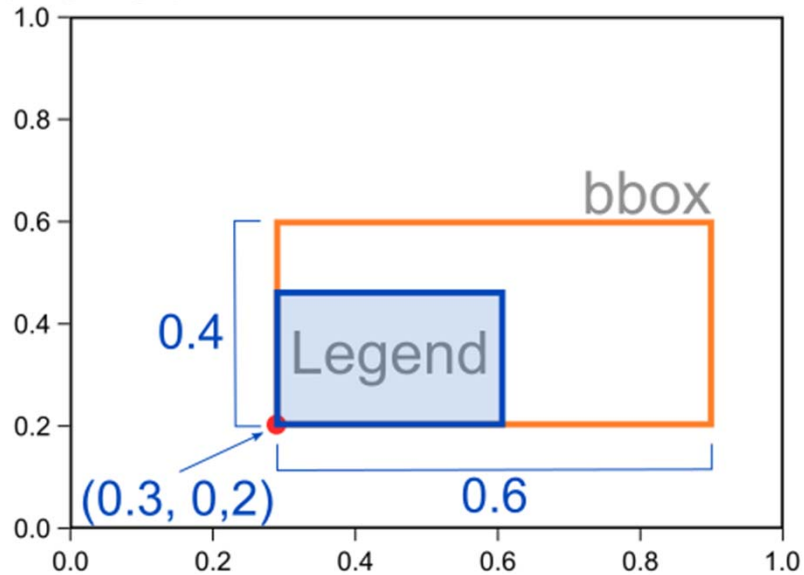


3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**

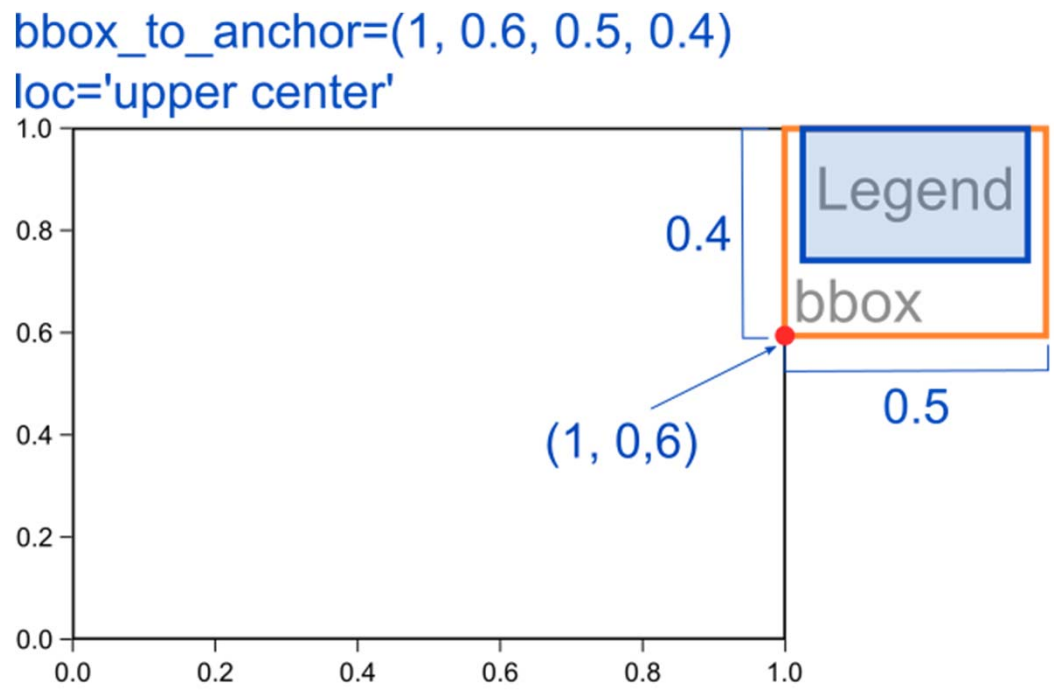
bbox_to_anchor=(0.3, 0.2, 0.6, 0.4)
loc='lower left'



3. Formatting

- Grids
- Labels
- Spines
- Scale
- Limits
- Ticks
- Twin axes
- **Legend**

- Legend position **bbox_to_anchor=(x, y, width, height)**



- **Let's see some examples:**

  03_Formatting/legend.ipynb

4. 2D plots

- **Scatter**
- Bar
- Pie
- Imshow
- Contour

- Scatter plots are used to plot **data points** on **x** and **y** axis in the attempt to show how much one variable is affected by another.
- A third variable can be set to correspond to the **color** or **size** of the markers, thus adding yet another dimension to the plot.
- Signature of Axes.scatter() function:
  - **Axes.scatter(x,y,s,c,cmap)**
- **Let's see some examples:**

  04_2D_plots/scatter.ipynb

4. 2D plots

- Scatter
- **Bar**
- Pie
- Imshow
- Contour

- Bar charts present categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.
- One axis of the chart shows the specific categories being compared, and the other axis represents a measured value.
- Signature of Axes.bar() function:
  - **Axes. bar(x, height, width, bottom, align)**
- **Let's see some examples:**

  04_2D_plots/bar.ipynb

4. 2D plots

- Scatter
- Bar
- **Pie**
- Imshow
- Contour

- A Pie Chart can only display one series of data. Pie charts show the size of items (called wedge) in one data series, proportional to the sum of the items. The data points in a pie chart are shown as a percentage of the whole pie.

- Signature of Axes.pie() function:

  - **Axes. pie(x, labels, colors, autopct)**

  autopct is used to label the wedges with their numeric value. The label will be placed inside the wedge. The format string will be fmt%pct.

- **Let's see some examples:**

  04_2D_plots/pie.ipynb

4. 2D plots

- Scatter
- Bar
- Pie
- **Imshow**
- Contour

- Display data as an image, i.e., on a 2D regular raster.
- The input may either be actual RGB(A) data, or 2D scalar data, which will be rendered as a pseudocolor image.
- For displaying a grayscale image set up the color mapping using the parameters cmap='gray', vmin=0, vmax=255.
  - **Axes. imshow(x, cmap, vmin, vmax)**
- **Let's see some examples:**

  04_2D_plots/imshow.ipynb

4. 2D plots

- Scatter
- Bar
- Pie
- Imshow
- **Contour**

- Contour plots (sometimes called Level Plots) are a way to show a three-dimensional surface on a two-dimensional plane.
- It graphs two predictor variables X Y on the y-axis and a response variable Z as contours. These contours are sometimes called the z-slices or the iso-response values.
  - **Axes.contour(x, y, z, levels)**
  - **Axes.contourf(x, y, z, levels)**
- **Let's see some examples:**

  04_2D_plots/contour.ipynb

5. 3D plots

- **Contour**
- Wireframe
- Surface

- The **Axes3D.contour3D()** function creates three-dimensional contour plot. It requires all the input data to be in the form of two-dimensional regular grids, with the Z-data evaluated at each point.
- Axes object need to be 3D (`projection='3d'`)
  - **Axes3D.contour3D(x, y, z, levels)**
- **Let's see some examples:**

  05_3D_plots/contour.ipynb

5. 3D plots

- Contour
- **Wireframe**
- Surface

- Wireframe plot takes a grid of values and projects it onto the specified three-dimensional surface, and can make the resulting three-dimensional forms quite easy to visualize. The **Axes3D.plot_wireframe()** function is used for the purpose.

- Axes object need to be 3D. (`projection='3d'`)

  - **Axes3D.plot_wireframe(x, y, z, rstride, cstride)**

- **Let's see some examples:**

  05_3D_plots/wireframe.ipynb

5. 3D plots

- Contour
- Wireframe
- **Surface**

- Surface plot shows a functional relationship between a designated dependent variable (Y), and two independent variables (X and Z). A surface plot is like a wireframe plot, but each face of the wireframe is a filled polygon.
- This can aid perception of the topology of the surface being visualized. The **plot_surface()** function x,y and z as arguments.
- Axes object need to be 3D. (`projection='3d'`)
  - **Axes3D.plot_surface(x, y, z, rstride, cstride)**
- **Let's see some examples:**

  05_3D_plots/surface.ipynb

6. Save plots

➢ Save plots

- The figures created can be saved to our local machines in different formats using **pyplot.savefig()** method.
  - **pyplot.savefig(fname, dpi, format, facecolor, edgecolor)**
- **Let's see some examples:**

  06_Save_plots/save_plots.ipynb

جامعة خليفة
Khalifa University

Thank You

ku.ac.ae